Chapter 4

Local Search Algorithms & Optimization Problems

- The searching Algos we have seen so far are only designed to explore the state spaces systematically (keeps one or more nodes in memory and *record* which alts have been explored at each point along the path)
- In **Local Search Algorithms** however , it only searches for the **final state** and not the path to get there .
    - *Example* : 8-queens puzzle , we only care about the final state of the board where no queens attack each other and not the path from the initial state to the goal state .

What is Local Search ?

Instead of exploring multiple paths or keeping track of the entire path taken (like in systematic search algorithms), local search algorithms focus on the **current state** (or "current node") and its immediate **neighbors**.

Advantages of Local Search

1. they use very little memory since they only need to store the current node and its neighbors .
2. they often find reasonable solutions in large/infinite state spaces for which systematic algos r not suitable .
3. useful for solving pure **Optimization Problems** (LP(maximizing profit given limited resources))

Limitations of Local Search

1. **Local Optima** : might get stuck at a local maximum
2. **No Guarantee of Global Optimum** it doesn't always find the global maximum !

State Space Landscape

- Think of the state space as a landscape with hills and valleys



where :

- **Elevation** represents the value of the **objective function / heuristic cost function**(measure we take to decide which neighbor to go to , *example* : height of the mountain to reach the highest mountain)
  - 🔺 if we use cost function then our aim woudl be to minimize it (**global min**)
  - 🔺 if we use the objkective function then our aim woukd be to maximize it (**global max**)
- **Global Maximum** : the highest peak in the entire landscape (maximizes the objective function)
- **Local Maximum** : A peak that is higher than it immediate neighbrs but no necessarily the higjest in the entire landscape.

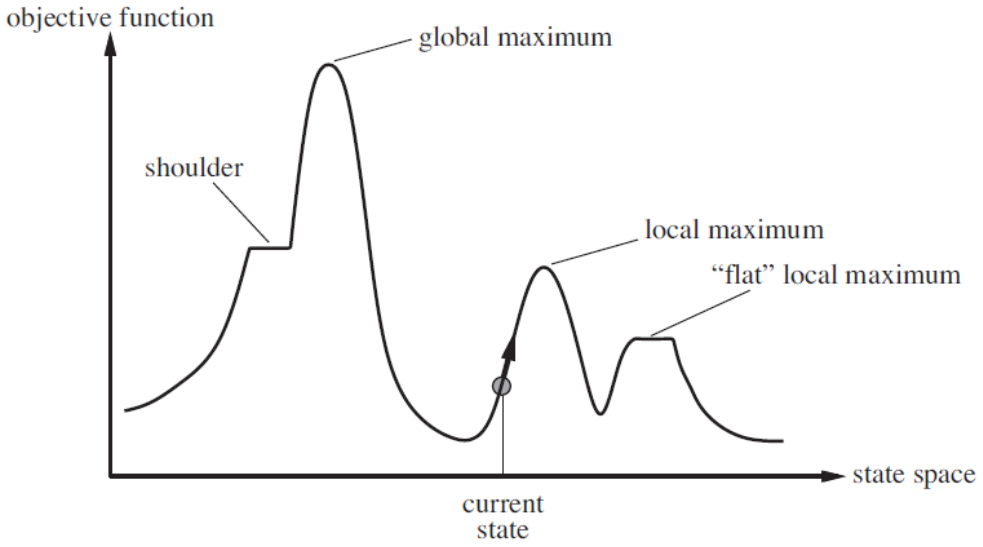


Figure 4.1 A one-dimensional state-space landscape in which elevation corresponds to the objective function. The aim is to find the global maximum. Hill-climbing search modifies the current state to try to improve it, as shown by the arrow. The various topographic features are defined in the text.

🌲 *Example* :

Example: Finding the Best Solution

Let's say you're trying to find the best solution to a problem, like maximizing profit in business:

1. **Current State**: Your current business strategy.
2. **Neighbors**: Slightly different strategies (e.g., changing prices, marketing tactics).
3. **Objective Function**: Profit. You want to maximize it.
4. **Process**: You evaluate each neighboring strategy to see if it increases profit. If it does, you adopt that strategy and repeat the process.

1. Hill Climbing search

1. Steepest ascent version

- hill climbing algorithmc continously moves in the direction of an increasing value (best sucessor) , if there's *more than one sucesssor* , then pick one at random , it terminates when it reaches the peak(global maximum)

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
    current ← MAKE-NODE(problem.INITIAL-STATE)
    loop do
        neighbor ← a highest-valued successor of current
        if neighbor.VALUE ≤ current.VALUE then return current.STATE
        current ← neighbor
```

Figure 4.2 The hill-climbing search algorithm, which is the most basic local search technique. At each step the current node is replaced by the best neighbor; in this version, that means the neighbor with the highest VALUE, but if a heuristic cost estimate h is used, we would find the neighbor with the lowest h.

Greedy Local Search :

- HC is sometimes called greedy local search cuz it looks for immediate improvement without thinking where to go next , but it often makes **rapid progress** towards the goal

Problems of Hill Climbing

1. Local Maxima :



- a state that is better than all its neighbors but not the best overall

2. Ridges : (heights as if)

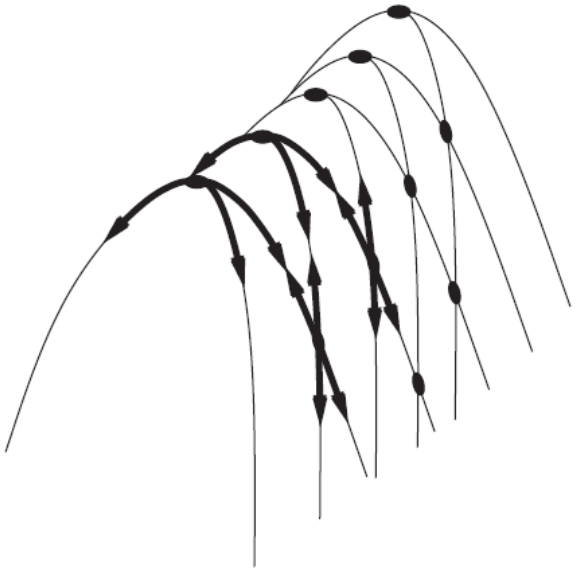


Figure 4.4 Illustration of why ridges cause difficulties for hill climbing. The grid of states (dark circles) is superimposed on a ridge rising from left to right, creating a sequence of local maxima that are not directly connected to each other. From each local maximum, all the available actions point downhill.



- A sequence of local maxima that makes it **HARD** to find the global maximum

3.Plateaus :



- flat areas where all neighbors have the same value , thid which may cause the algorithm to get **STUCK**

> **Remark**
>
> - Hill Climbin is **Incomplete** beacuse of local maxima , there might exist a solution but it can *not find it*

Improvements of Hill-Climbing

1. Sideways Moves :

- we allow the algorithm to move to a neighbor with the **same value** (this helps escape plateaus)

⭕**BUT** , there's a risk of falling into an infinte loop ! in caqse of a real plateau
✅**SOLUTION** : we can limit the number of consecutive sideways moves allowed
Example :
we could allow up to, say, 100 consecutive sideways moves in the 8-queens problem. This raises the percentage of problem instances
solved by hill climbing from 14% to 94%. Success comes at a cost: the algorithm averages roughly 21 steps for each successful instance and 64 for each failure

Variants of Hill-Climbing

1. Stochastic Hill Climbing

- instead of always choosing the best neighboring state , stochastic HC randomly selects **any** neighborig state that improves the current state
- It adds randomness, which can help escape **local maxima** (suboptimal solutions) and explore more of the search space.
- It's slower than steepest-ascent HC because it doesn't always choose the best move, but it can sometimes find better solutions in complex landscapes.

2. First-Choice Hill-Climbing

- This is a more efficient version of stochastic HC , instead of evaluating all neighbors , it randomly generates neighbors one by one and stops as soon as it find a better state .
- it's useful when there are **thousands of neighbors**

3. Random Restart Hill Climbing

- if HC gets stuck (e.g at a local maximum) it starts over from a new random state , This process is repeated until a goal state is found
- If each HC search has a probability p of finding a solution, the expected number of restarts needed is $1/p$

Simulated Annealing (SA)

Hill climbing always moves uphill (toward better solutions) and never moves downhill (towards worse solutions) , which makes it **incomplex** , it can get stuck at a *local maximum*

> **note**
>
> Imagine you're climbing a mountain, and you reach a small hill. Hill-climbing would stop there because it can't go downhill to explore other, higher peaks. But what if the tallest mountain is just beyond that hill?

and the Problem with Random Search although is **complete** but is extremly **inefficient**

SA combines the best of both worlds , here's the steps for the simulated annealing

How it's implemented :

1. **Start with a Random State**: Begin at a random solution (e.g., a random arrangement of queens in the 8-queens problem).
2. **Pick a Random Neighbor**: Instead of always choosing the best neighbor (like hill-climbing), pick a random neighbor.
3. **Decide Whether to Move**:
   - If the neighbor is **better**, always move there.
   - If the neighbor is **worse**, sometimes move there anyway. The probability of accepting a worse move depends on:
     - **How much worse** the move is (smaller downhill steps are more likely to be accepted).
     - The **temperature** (a parameter that decreases over time). At *high* temperatures, *worse* moves are more likely to be accepted. As the temperature *cools*, worse moves become *less* likely.
4. **Repeat**: Keep repeating this process, gradually lowering the temperature.

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
          schedule, a mapping from time to "temperature"

  current ← MAKE-NODE(problem.INITIAL-STATE)
  for t = 1 to ∞ do
      T ← schedule(t)
      if T = 0 then return current
      next ← a randomly selected successor of current
      ΔE ← next.VALUE − current.VALUE
      if ΔE > 0 then current ← next
      else current ← next only with probability e^(ΔE/T)
```

Figure 4.5 The simulated annealing algorithm, a version of stochastic hill climbing where some downhill moves are allowed. Downhill moves are accepted readily early in the annealing schedule and then less often as time goes on. The *schedule* input determines the value of the temperature T as a function of time.

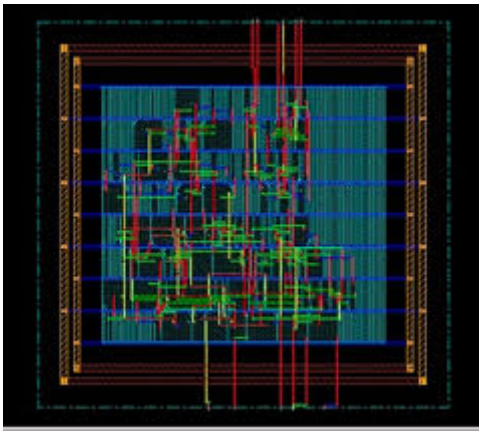
visualisation : Visualise

Why does simulated Annealing work ?

- at High Tempertaure , the algorithm behaves like a random search , exploring the search space widely and **escaping** local maxima.
- as it cools it behaves more liek hill cliumbing search and By the time the temperature is very low, the algorithm is likely to have found a **global optimum** (the best solution).

Uses :

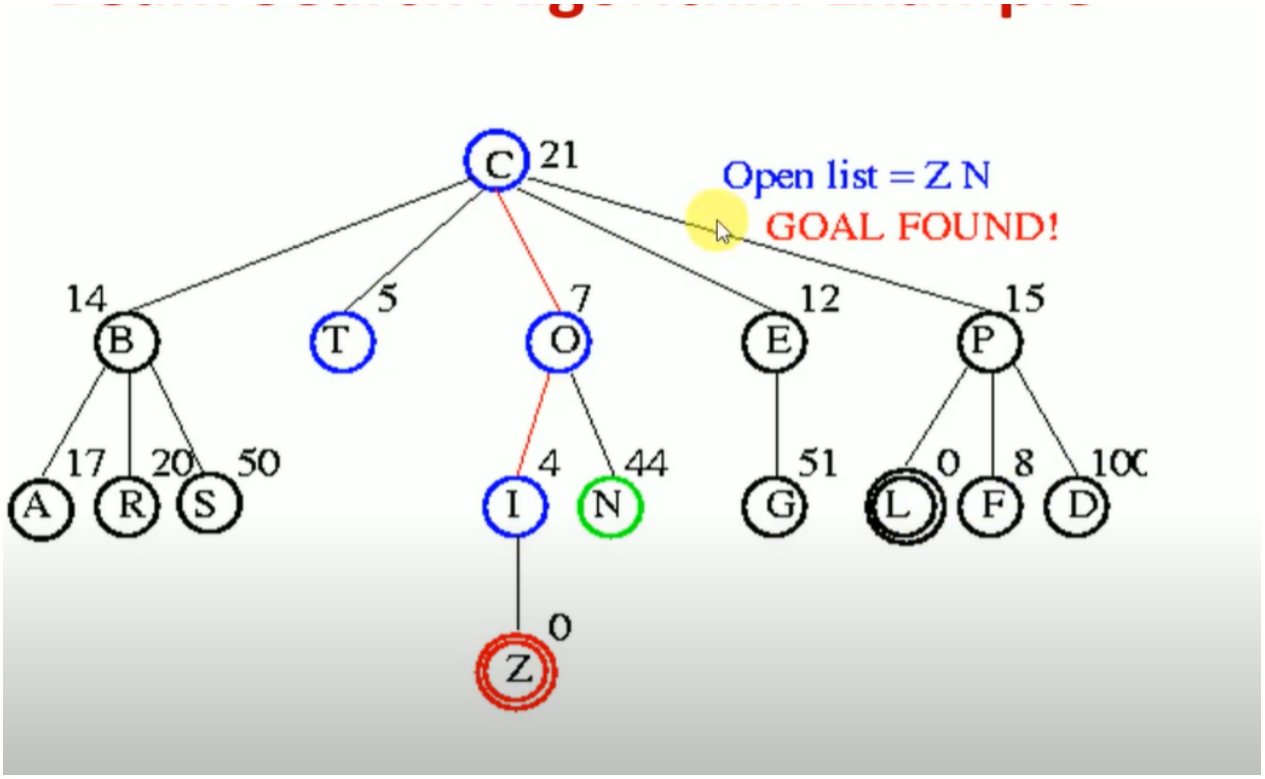
- **VLSI Layout**: Designing efficient layouts for computer chips.



- **Factory Scheduling**: Optimizing schedules for manufacturing processes.
- **Traveling Salesman Problem**: Finding the shortest route for a salesman visiting multiple cities.

Local Beam Search

- This search algorithm keeps track of multiple states , it extends the most promising node in a limited set (node with the least heuristic function)
- *example*:



    - here we set the n (maximum numbr of nodes on the node list to be 2) , then we choose the bone with the least cost function

1. **Start with k Random States**: Instead of starting with just one random state (like in hill-climbing), Local Beam Search starts with **k random states**. For example, if $k = 5$, you start with 5 random arrangements of queens in the 8-queens problem.
2. **Generate Successors**: For each of the k states, generate all possible successors (neighboring states). For example, in the 8-queens problem, each state has 56 successors (since each queen can move to 7 other squares in its column).

3. **Select the Best k Successors !**: From all the successors generated, pick the **k best ones** (based on the objective function, e.g., the number of attacking pairs in the 8-queens problem). (*example* best 2s (k=2 then we expand them both))
4. **Repeat**: Keep repeating this process until a goal state is found (e.g., a state with 0 attacking pairs in the 8-queens problem).

Stochastic Beam Search (SBS)

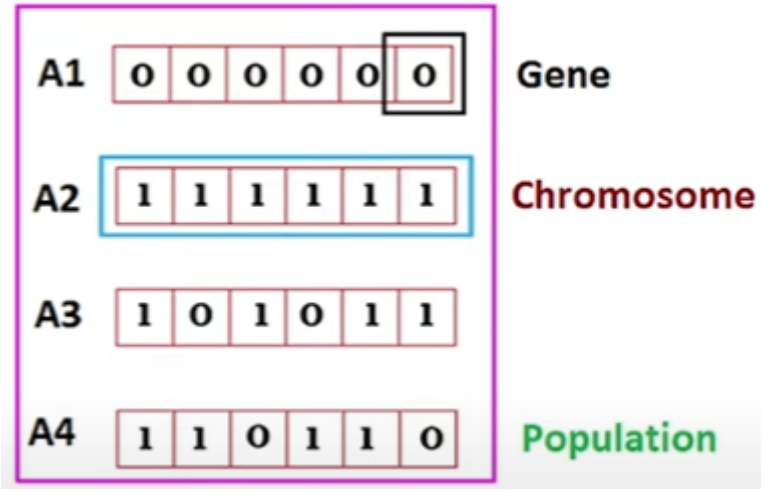
- instead of choosing the best k from the pool of candidates sucvessors , SBS chooses k sucessors at random , with the probabaility of choosihg a given sucessor being an increasing function of the value of thid sucessor
- it **randomly** selects candidates with a higher probability of choosing better ones. This helps maintain **diversity** and avoids getting **stuck** in one area.

Genetic Algorithm :

- a variant of SBS , where the sucessor states are generated by combining two parent states rather than modifying a single state .

How it's implemented :

1. k randomly generated sucessors **The Population** , each state/ individual / **chromosome** is a solution to the problem and is represented by a string over a finite alphabet (most commionly 0s and 1s) (each one is **Gene**)



2. **Individulas selection**
   - **fitness Function** (objective function)
     - a function that returns higher values for the better individuals .(how close the indiv is to the goal)
     - *example* : in 8-queens problem the fitness function can be “the number of non-attacking pairs of queens”
- Select individuals to "**reproduce**" based on their fitness scores. (with some probability) Individuals with higher fitness scores have a better chance of being selected.

Remark

- when we select the fittest individual we actually use 3 different methoids but that is not covered in the course :

1. Roulette wheel selection

2. Tournament selection
3. Rank-based selection

3.**CrossOver**:

- we combine the two selected indivs to create one or more offsprings , this is done by choosing **crossover point** and swapping the parent's strings !

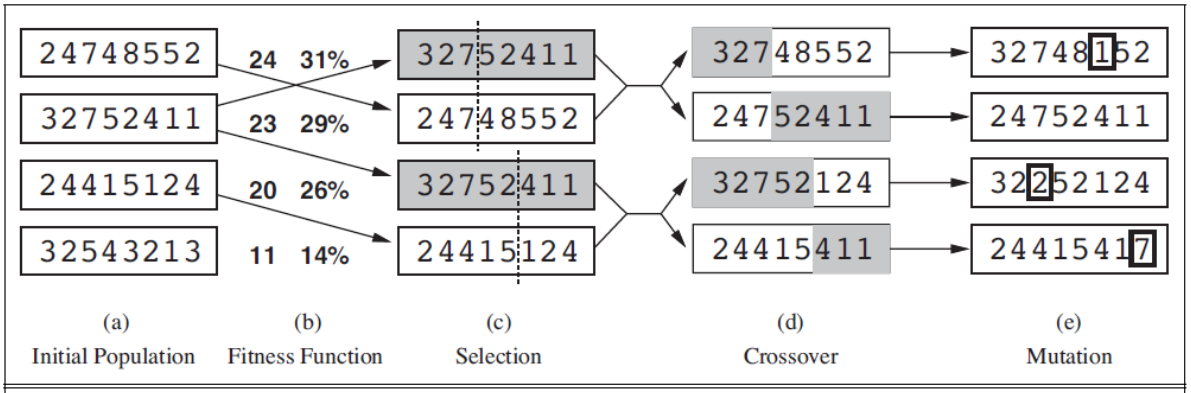


Figure 4.6 The genetic algorithm, illustrated for digit strings representing 8-queens states. The initial population in (a) is ranked by the fitness function in (b), resulting in pairs for mating in (c). They produce offspring in (d), which are subject to mutation in (e).

4.**Mutation** :

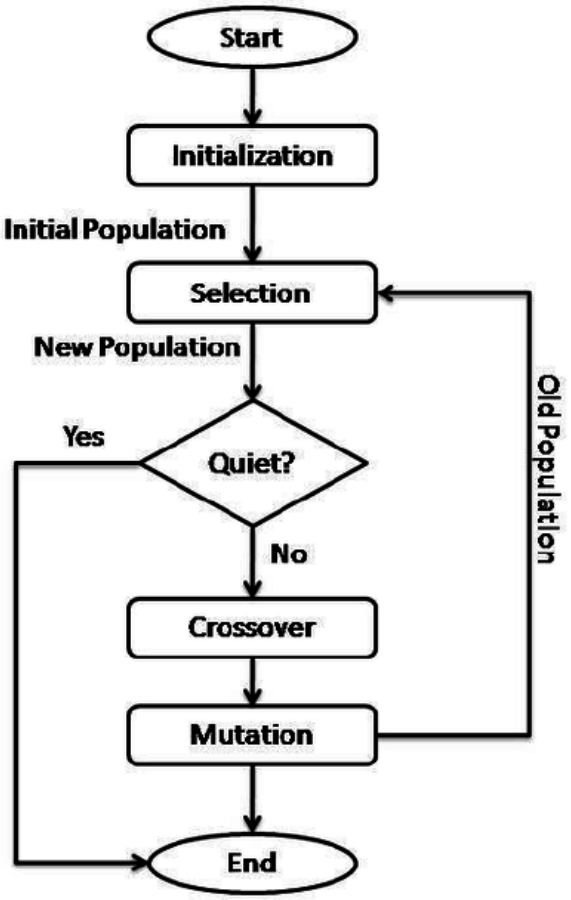
- Introduce small random changes to the offspring to maintain **diversity** in the population. This is done by flipping a bit or changing a digit with a small probability.

> **Remark**
>
> - mutation is also only done for specifici individulas in specific cases (with low radnom probability) using Diff Methods
>   - Flip bit
>   - Gaussian mutation
>   - exchange/swap mutation

5. **New Pop Gneration** (elitism)

- replace the old population with the new population of offspring (and possibly some of the best individuals from the previous generation, depending on the implementation).
- Repeat the process: evaluate fitness, select, crossover, and mutate. (*a pair*)

5. **Termination**:
   - we stop when a satisfactory solution is found (an indiv with perfect fitness score) or after a certain number of generations


Start
Initialization
Initial Population
Selection
New Population
Yes
Quiet?
No
Crossover
Mutation
End
Old Population

Pseudo Code

```
function GENETIC-ALGORITHM(population, FITNESS-FN) returns an individual
  inputs: population, a set of individuals
          FITNESS-FN, a function that measures the fitness of an individual

  repeat
      new_population ← empty set
      for i = 1 to SIZE(population) do
          x ← RANDOM-SELECTION(population, FITNESS-FN)
          y ← RANDOM-SELECTION(population, FITNESS-FN)
          child ← REPRODUCE(x, y)
          if (small random probability) then child ← MUTATE(child)
          add child to new_population
      population ← new_population
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to FITNESS-FN
```

```
function REPRODUCE(x, y) returns an individual
  inputs: x, y, parent individuals

  n ← LENGTH(x); c ← random number from 1 to n
  return APPEND(SUBSTRING(x, 1, c), SUBSTRING(y, c + 1, n))
```

Figure 4.8 A genetic algorithm. The algorithm is the same as the one diagrammed in Figure 4.6, with one variation: in this more popular version, each mating of two parents produces only one offspring, not two.